

Implement Client Script & UI Policy (Incident)

SCENARIO	ENTER			ENGAGE		EXIT	EXTEND			
Creating, updating, and submitting an Incident while dynamic Client Scripts and UI Policies guide the form.	Open the Incident form and begin entering information.			Core form interactions apply rules, validations, and dynamic field behavior.		Submit the Incident after required conditions are satisfied.	Update, review, and manage the Incident after submission.			
Steps What does the person (or group) typically experience?	Open Incident User opens the Incident form from ServiceNow.	Enter details User enters short description, caller, category and other details.	Dynamic behavior Client Script and UI Policy react to field changes and control behavior.	Validation Required/conditional fields and validation prevent incomplete or incorrect data.	Submit User submits the Incident after completing required information.	Confirmation The Incident is created and the user can see the saved record.	Re-open / edit User opens an existing Incident to review or modify it.	Change field Changing a field triggers the configured dynamic behavior again.	Save update User saves the updated Incident.	Review result User verifies the final field values and record state.
Interactions What interactions do they have at each step along the way?	ServiceNow Incident module; form and navigation.	Incident fields, reference lookups, choice lists and text inputs.	Client-side form logic; UI Policy actions; field visibility/read-only/mandatory states.	onSubmit/onChange logic and UI Policy conditions.	Submit action and browser-side checks.	Incident record, confirmation state and list/form views.	Existing Incident form and record data.	Field changes trigger scripts/policies.	Save action and updated record.	Incident form/list and final values.
Goals & motivations At each step, what is a person's primary goal or motivation? ("Help me..." / "Help me avoid...")	Help me reach the correct Incident form quickly.	Help me enter complete and relevant information.	Help me understand what the form expects without confusion.	Help me avoid submitting invalid or incomplete data.	Help me create the Incident with confidence.	Help me know the record was saved successfully.	Help me review the record before changing it.	Help me keep related fields consistent automatically.	Help me save updates without losing information.	Help me confirm the automation behaved as intended.
Positive moments What steps are enjoyable, productive, motivating, or reassuring?	Clear Incident entry point.	Structured fields make data entry predictable.	Fields respond dynamically to user actions.	Validation catches issues before submission.	Successful submission completes the task.	Saved record gives immediate confirmation.	Existing data is easy to review.	Automatic field behavior reduces manual work.	Updates are stored in the record.	Final verification shows the rules working.
Negative moments What steps are frustrating, confusing, costly, or time-consuming?	Finding the correct module can take time.	Too many fields can feel repetitive.	Unexpected field changes may confuse users.	Validation messages can interrupt the flow.	A blocked submit can be unclear without feedback.	Users may need to verify several fields manually.	Editing can expose the same dynamic rules again.	Incorrect conditions can cause unwanted behavior.	Unsaved changes can be lost if navigation happens.	Testing every rule combination takes time.
Areas of opportunity How might we make each step better? What ideas do we have?	Use clear module navigation and favorites.	Keep only necessary fields prominent.	Use simple, consistent UI Policy conditions.	Give useful client-side messages and validation.	Use onSubmit checks only where necessary.	Show clear confirmation and record state.	Make editable/read-only states predictable.	Test onChange behavior for all relevant values.	Warn users before leaving with unsaved changes.	Create a small test checklist for each rule.